

Interactive GUI Application for Machine Learning Models

Project Report

SECOND YEAR
GROUP PROJECT
GROUP 10

**FACULTY OF COMPUTING AND TECHNOLOGY
UNIVERSITY OF KELANIYA.
B.Sc. (Honors) in Computer Science**

May 2025

Supervisor

Dr. Ishan Sabar

Group Members

Student Number	Student Name
CS/2021/038	Jayawardena K.P.A.P.T
CS/2021/008	Galappaththi S.H.A
CS/2021/005	Mihirud K.M.M
CS/2021/062	Rajawardhane S.R.P.D.D
CS/2021/024	Wickramathilaka J.O.R
CS/2021/025	Induwara K.G.R

Abstract

Machine Learning (ML) and Data Science are rapidly growing fields, yet their complexity often objects especially for beginners with limited coding experience to enter the ML field. Our project aims to introduce an interactive, cross-platform desktop application designed to pave a pathway to ML education by offering a fully graphical, no-code environment for building, training, and evaluating ML models. The primary objective is to create a user-friendly interface that enables students and educators to explore ML workflows through drag-and-drop modules and visualizations, without requiring prior programming knowledge.

The application supports data preprocessing, feature engineering, model training, performance evaluation, and prediction analysis using open-source libraries such as PyTorch, Scikit-learn, and NumPy. The intended application aims to offer educational value through real-time graphical explanations of ML concepts, making it useful for classroom settings and self-learners. The system architecture includes a React.js combined with an Electron.js frontend and a python backend, ensuring responsiveness and modular design. By connecting intuitive user experience with key educational insights, the platform bridges the gap between theory and practical understanding, influencing users to confidently engage with modern ML concepts in a hands-on manner.

Acknowledgement

This project has been a collaborative effort, and special gratitude is extended to all team members for their contributions. The collaborative efforts of **Jayawardena K.P.A.P.T, Galappaththi S.H.A, Mihirud K.M.M, Rajawardhane S.R.P.D.D, Wickramathilaka J.O.R** and **Induwara K.G.R** were essential in completing the project successfully.

Sincere gratitude is extended to our supervisor, **Dr. Mohamed Ishan Sabar**, for the continuous support, patience, and extensive knowledge that were instrumental throughout the research and writing of this report.

Appreciation is also expressed to the faculty members of the Faculty of Computing & Technology, particularly **Dr. Navodi Mekhala Hakmanage**, for their insightful comments and encouragement, which contributed to broadening the research from various perspectives.

Sincere thanks are also extended to colleagues for their assistance with various aspects of the research. Appreciation is also extended to all contributors, whose support was crucial to the project's completion.

Table of Contents

1. Introduction	8
1.1 Motivation for the project	8
1.2 Project Objectives	9
1.3 Scope of the Project	10
1.4 Limitations	12
2. Background	12
2.1 Problem Identification	12
2.2 Literature Review	13
2.3 Methods and Tools	14
3. Design	16
3.1 Overall Design	16
3.2 Selected sections to be implemented	17
4. Implementation	27
4.1 Technologies used	27
4.2 User Interface Design	27
4.3 Reports Generated	28
5. Discussion and Conclusion	28
5.1 Discussion on the process followed during the project	28
5.2 Conclusions drawn	29
6. Future Work	30
7. References	31

List of Figures/ Tables

Figure 1 - Example image for a neural network with 2 hidden layers	9
Figure 2 - Basic architecture of the solution	27
Figure 3 - Data Importing & Preprocessing Module	32
Table 1 – Backend Components of the application	25

List of Abbreviations

Artificial Intelligence	AI
Machine Learning	ML
Artificial Neural Network	ANN
Convolutional Neural Network	CNN
Graphical User Interface	GUI
Exploratory Data Analysis	EDA
Recurrent Neural Network	RNN
Long Short-Term Memory	LSTM
Application Programming Interface	API
Stochastic Gradient Descent	SGD
Support Vector Regression	SVR
K-Nearest Neighbours	KNN
Support Vector Machine	SVM
Receiver Operating Characteristic	ROC
Locally Linear Embedding	LLE
Multidimensional Scaling	MDC
Principal Component Analysis	PCA
Independent Component Analysis	ICA
Uniform Manifold Approximation and Projection	UMAP

1. Introduction

In the early 2010s, machine learning emerged as a transformative approach to solving complex problems, laying the groundwork for modern artificial intelligence. Over time, the field of AI has evolved beyond traditional machine learning algorithms, which were primarily statistical, toward more specialized and abstract methodologies. However, despite these advancements, traditional machine learning remains an essential starting point for those new to AI, as the complexity of modern machine learning pipelines and workflows can be overwhelming for beginners. Our project aims to bridge this gap by providing a user-friendly platform that abstracts away the complexities of coding, offering a simple graphical interface for users to interact with machine learning models. Developing such a platform requires extensive research into both fundamental machine learning concepts and existing tools. Therefore, reviewing relevant literature is a crucial step in guiding the design and implementation of our application.

NEURAL NETWORK WITH TWO HIDDEN LAYERS

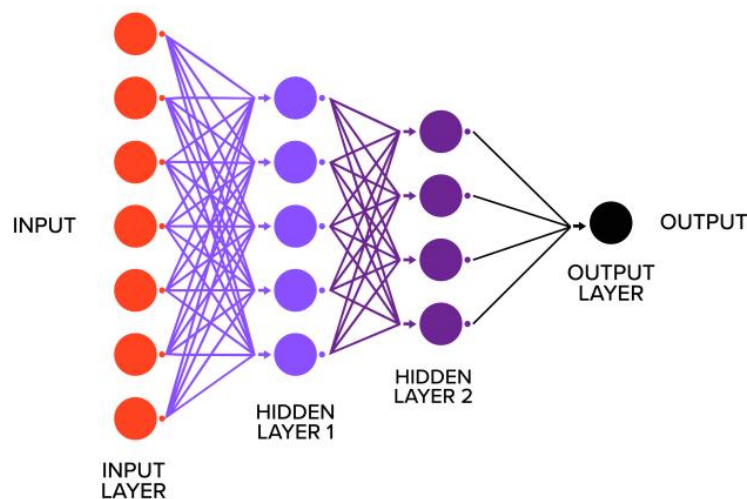


Figure 1 An example image for a neural network with 2 hidden layers

1.1 Motivation for the project

The motivation for initiating this project arose from a clear and pressing gap in the current educational landscape of machine learning (ML). As machine learning becomes increasingly relevant across industries, there's a growing interest among students, educators, and self-

learners to understand its fundamentals. However, most existing ML tools and platforms are either heavily code-centric or designed with enterprise users in mind, making them unsuitable for those at the beginning of their learning journey.

While platforms like Microsoft Azure ML Studio, Google AutoML, and IBM Watson provide powerful capabilities, they are primarily built for business-driven solutions and often prioritize deployment and performance over conceptual clarity. As a result, they can be overwhelming for users who are not yet familiar with programming, data preprocessing, or model evaluation techniques. On the flip side, beginner-friendly tools like Google Teachable Machine offer simplicity but lack the depth needed to explore the full ML pipeline — such as understanding feature engineering, exploring multiple model types, and evaluating performance with meaningful metrics.

We identified a missing middle ground, a platform that is simple enough for beginners to use without coding, yet deep enough to allow them to meaningfully interact with the different stages of a machine learning workflow. This insight became the foundation for our project: to build an educationally focused, intuitive GUI-based application that enables users to explore, build, and evaluate machine learning models visually.

By eliminating the need for programming and instead focusing on visual interactivity and conceptual clarity, our project aims to make ML education more accessible, engaging, and hands-on. This tool is especially valuable in academic environments, where students can benefit from experimenting with real-world datasets and ML pipelines, all within a supportive, no-code interface that enhances understanding rather than overwhelming with complexity.

1.2 Project Objectives

1.2.1 Develop an Intuitive User Interface

- Create a visually intuitive GUI using React to enable users to build machine learning models without coding.

1.2.2 Implement Comprehensive Preprocessing Tools

- Design a powerful preprocessor module to allow users to perform feature engineering tasks such as cleaning, encoding, filtering, Grouping, and transforming datasets interactively.

1.2.3 Provide Modular ML Functionalities

- Develop modules for:
- Building and configuring machine learning models (Model Builder).
- Testing and analysing model performance.
- Making predictions and interpreting outcomes with user-provided data (Inference Engine).

1.2.4 Integrate Educational Features

- Use real-time visualizations (data distributions, decision boundaries, model performance graphs, hidden weights and biases, kernel tensors) to explain ML concepts.
- Highlight the reasoning behind preprocessing steps, model selection, and performance evaluation metrics.

1.2.5 Utilize Open-Source Libraries Effectively

- Leverage sklearn, NumPy, pytorch, and scipy for computations, and matplotlib and seaborn for data visualization.

1.2.6 Optimize for Beginner Accessibility

- Provide guides, tutorials and tooltips for users unfamiliar with machine learning on how to use the application.

1.2.7 Conduct Testing and Validation

- Test the application rigorously with target users (students, educators, and ML enthusiasts) to ensure functionality, usability, and educational value.

1.3 Scope of the Project

The main scope of the project is to build a functioning code free machine learning Graphical User Interface application which would be useful for both new learners and anyone who wants to test statistical data. The application will let the user test a selection of a machine learning models and this will also include a mode to create a neural network in a GUI and test it as well.

Below are the primary aspects within the scope of the project:

- **User Interface Development**

Design and develop a drag-and-drop style graphical user interface (GUI) using Electron.js, allowing users to construct machine learning workflows without writing code.
- **Data Handling and Preprocessing**

Implement a data frame manager and preprocessor module that enables users to import datasets and perform essential preprocessing tasks such as filtering, grouping, encoding, transformation, and normalization visually.
- **Machine Learning Model Integration**

Provide support for a variety of machine learning algorithms (e.g., classification, regression models) through a Model Builder module. Users should be able to select models, configure hyperparameters, and train them on uploaded datasets.
- **Evaluation and Inference**

Include an Inference Engine to test model predictions with new data inputs, along with detailed performance metrics such as accuracy, confusion matrix, and visual charts like ROC curves and bar charts for class distribution.
- **Educational Visualization**

Use real-time visualizations (such as decision boundaries, data distributions, and training curves) to support conceptual understanding. These should be interactive and designed to explain ML concepts rather than just output numbers.
- **Cross-Platform Deployment**

Ensure that the application is compatible with Windows, macOS, and Linux systems, and can be packaged for easy installation on each.
- **Open-Source Foundation**

The application relies on widely used, open-source Python libraries such as Scikit-learn, PyTorch, NumPy, SciPy, Matplotlib, and Seaborn, allowing for both transparency and extensibility.
- **Documentation and Support**

Create user guides, tutorials, and embedded tooltips to help first-time users understand how to use the application and gain basic ML knowledge.
- **Testing and User Feedback**

Perform functional, usability, and integration testing. Collect feedback from target

users such as students, educators, and ML enthusiasts to refine usability and educational effectiveness.

1.4 Limitations

One key limitation we identified in existing AutoML software is the lack of comprehensive support for neural networks. Recognizing this as a critical gap, we designed our platform to include support for artificial neural networks (ANNs), allowing users to experiment with deep learning alongside traditional machine learning models. Other limitations we faced could be listed as well.

- **Increased System Complexity**

Integrating artificial neural networks significantly increased the complexity of the backend logic due to the need for layer configuration, activation functions, backpropagation, and training loop control.

- **Conceptual Depth**

Neural networks require deeper understanding of gradient descent, weight updates, and hyperparameter sensitivity, making it difficult to present in a simplified, beginner-friendly interface without oversimplifying key concepts.

- **Visualization Challenges**

Visualizing the training process (like loss curves, hidden layer activations, etc.) in real time while keeping the UI responsive proved technically demanding, especially for larger datasets or deeper models.

- **Limited Hyperparameter Tuning**

Providing a no-code yet flexible interface for neural network configuration (e.g., optimizer choice, learning rate, batch size) was limited to a simplified version, restricting more advanced users.

2. Background

- **Problem Identification**

Mainly there was a gap between new learners and professionals when interacting with machine learning models. Machine learning models are commonly used via complex coding, thus making it a problem for the fresh newbies who are in to machine learning. Since they don't have the exact clear knowledge on how to interact with a machine learning model, train suitable machine learning models for various types of data sets and the part of dealing with complex code, these new learners, users and student would have to face difficulties when interacting with ML models

• Literature Review

Our project aims to bridge the gap between new learners and professionals who are into machine learning field by providing a user-friendly platform that abstracts away the complexities of coding, offering a simple graphical interface for users to interact with machine learning models.

In our project, we focus on providing machine learning tools rather than data analysis tools. Nevertheless, since data analysis is an essential part of the workflow, we integrate **Auto EDA (Exploratory Data Analysis) algorithms** and **feature importance measurement techniques** [1][2] to automate these tasks while maintaining user control.

While there are already AutoML solutions [3][4][13], they primarily address **Combined Algorithm Selection and Hyperparameter Optimization**. This issue has already been studied and solved [5], and we are implementing some of the techniques ensuring user have the relevant control over the ML process.

Our application design is inspired by existing ML platforms like **Orange3**[6] and **KNIME** [7]. So, the structure of the UI/UX is Userfriendly and functional [14][15]. We also reviewed tools such as **Netron** to enhance the model interpretability as well.

Also, we designed our platform to include support for **artificial neural networks** (ANNs), allowing users to experiment with deep learning alongside traditional machine learning models.

Our application is designed to work with tabular data, making it more accessible and easier for beginners to navigate and understand. Since Convolutional Neural Networks (CNNs) excel at detecting spatial features [16][17], which are primarily found in image data, we have opted not to include support for building CNN models.

Recurrent Neural Network (RNN) models, such as Long Short-Term Memory (LSTM) networks, are particularly well-suited for tasks involving sequential data, including time series prediction, speech recognition, and machine translation. Apart from time series analysis, many of these problems involve data that is not inherently tabular and therefore falls outside the scope of our application.

• **Methods and Tools**

Our project focuses on **machine learning frameworks**, **user interface technologies**, and **data processing tools**. Altogether the output application aims to build up a complete no-code ML application. The methods and tools we used throughout this project are listed down accordingly.

1. Exploratory Data Analysis & Feature Engineering

- **Pandas**: For tabular data manipulation, cleaning, transformation, and exploration.[9]
- **Scikit-learn**: Used for implementing core ML models (e.g., logistic regression, decision trees, SVMs), preprocessing (scaling, encoding), and evaluation metrics.[11]
- **PyTorch**: Provides functionality for building, training, and visualizing artificial neural networks (ANNs), including custom layer design and backpropagation.
- **SciPy**: Supports statistical computations and scientific analysis.[12]
- **NumPy**: Low-level matrix and array operations.[10]

2. Visualizations and GUI

- **Recharts, Plotly.js, and ApexCharts**: For generating interactive and educational visualizations such as:
 - Heatmaps
 - Used for showing the intensities or relationships between pair of values. A colour coded matrix could be used where brighter colors indicating a stronger relationship while darker colors representing a weaker relationship.
 - Could be used for exploring feature correlations or to visualize confusion matrices
 - Histograms

- This shows the distribution of a single feature of a data set. A bar plot where each of the bar representing the frequency of data points that fall into that specific range
- Could be used to check data distribution, spot skewness, outliers, or multi-modal patterns
- ROC curves
 - ROC curves are used as a method to evaluate how well a characteristic model distinguishes between classes. Used for binary classification and to compare classifier performance across thresholds
- Confusion matrices
 - Shows exact counts of predictions. Also, it could show which ones are correct vs which ones are misclassified. Shows a table of actual vs predicted classes. It could be True positives, False positives, True negatives, or False negatives.

3. Backend & API Layer

- **FastAPI:** Lightweight, asynchronous Python framework for connecting backend logic with the Electron-based **Frontend & GUI Framework** [8]
- **React.js:** Modular UI library used to build interactive, responsive components for data uploading, model configuration, training control, and visualizations.
- **Electron.js:** Converts the web-based React app into a cross-platform desktop app (Windows).

4. Development Tools and Workflows

- **Git + GitHub:** Version control and collaboration.
- **VS Code / PyCharm:** Primary development environments for frontend and backend teams.

5. Testing & Validation

Testing and validation are currently in progress to ensure the system performs reliably and accurately across various data scenarios. This phase involves evaluating the model's predictions using unseen datasets, fine-tuning parameters, and checking

for consistency, robustness, and generalization. Any bugs or performance issues identified are being addressed to finalize a stable and effective implementation.

6. Educational Enhancements

- **Tooltips / Contextual Help:** Every module is equipped with pop-up guides to explain ML terms like "regularization", "accuracy", or "data leakage".
- **Live Visualization Components:** Custom React components linked to backend updates for real-time training curves and model feedback.

3. Design

The design of our educational GUI-based ML application focuses on accessibility, modularity, and real-time feedback. The application is structured to offer a visual, no-code experience that mirrors real-world ML workflows, enabling users to build, train, and evaluate models without any programming knowledge.

3.1 Overall Design

The overall system is designed to provide a beginner-friendly application for users. It basically implements the following main components in the application

- **Frontend Interface:** Users can visually organize components such as Data Import, Preprocessing, Model Training, and Evaluation.
- **The model engine** supports various ML algorithms such as classification, regression, neural networks, etc., using Scikit-learn and PyTorch. It is responsible for training models, managing hyperparameters, and returning results for evaluation and prediction.
- **Preprocessor Module:** Clean, encode, normalize, and transform data visually. Supports common techniques such as one-hot encoding, min-max scaling, and null value handling.
- **Visualization Layer:** Provides real-time feedback via confusion matrices, ROC curves, loss graphs, and feature importance charts using libraries like Plotly.js and ApexCharts.
- **Data Frame Manager:** Offers users an interactive table to view, filter, and manipulate their dataset before modeling.
- **Inference Engine:** Enables prediction on new user inputs, with clear visual output and comparison across models.

- **Educational Support Features:** Integrated tooltips, glossary, and step-by-step guides explain ML concepts, making the tool suitable for classroom use and self-study.

3.2 Selected sections to be implemented (with justification)

3.2.1. User Interface Layer (Frontend with React + Electron)

- **Drag-and-Drop Workflow Builder**

A visual canvas where users can construct ML workflows by dragging modules like "Data Import", "Model Builder", etc.

- **Interactive Visual Feedback**

Real-time updates on model training, preprocessing results, and performance metrics.

- **Tooltip + Help System**

Beginner-focused tooltips and pop-ups explaining each feature or term

A User Interface will allow users to build ML workflows visually and intuitively without touching a line of code, ensuring thorough engagement from students and beginners

3.2.2 Data Frame Manager

- **Dataset Import & Display**

Load CSV files and preview data.

- **Data Manipulation Features**

- Column selection
- Sorting/filtering
- Basic stats like mean, mode, null count

Beginners must first learn to understand and explore datasets before modelling. The data frame manager simplifies this by offering visual previews and basic operations (e.g., sorting, column removal, null checks).

3.2.3 Preprocessor Module

- **Feature Engineering Tools**

- Cleaning null/missing values

- Handling missing or null values could be critical in machine learning model training because they won't function without proper complete set of data.
- One-hot encoding / Label encoding
 - Categorical data needs to be converted to numerical values for machine learning models to interpret them. Label encoding assigns a unique number to each category, while one-hot encoding creates a separate binary column for each of this category.
- Normalization & Standardization
 - Normalization transforms data in to [0,1] range using min-max scaling and standardization adjusts data to have a mean 0 and a standard deviation of 1
- Filtering, grouping, categorization
 - Step to cleaning up the data set by removing unnecessary data, summarizing data based on categories and converting continuous values into discrete categories.
- Transformation pipelines (e.g., log-transform, binning)
 - Transformation pipelines apply mathematical functions restructuring techniques to features to make the data more suitable analysis.

Data preprocessing is a critical step in ML and often a barrier for newcomers. This module will introduce concepts like missing value handling, encoding, and normalization through GUI controls

3.2.4 Model Builder

- **Model Selection**
 - Classification: Logistic Regression, Decision Trees, SVM, KNN
 - Regression: Linear Regression, Random Forest Regressor
 - Neural Networks: Basic ANN with user-defined layers
- **Hyperparameter Configuration**
 - Learning rate, batch size, epochs
 - Max depth, number of neighbours, etc.

This module lets users explore different algorithm types (e.g., classification vs regression) and experiment with hyperparameters.

3.2.5 Training and Evaluation

- **Training Process Handler**
Backend-triggered model training with progress updates.
- **Evaluation Metrics Display**
 - Accuracy, Precision, Recall, F1-score
 - Confusion matrix
 - ROC-AUC curve (for binary classification)
 - R^2 , MAE, MSE (for regression)

This section visualizes accuracy scores, confusion matrices, ROC curves, and regression metrics in real-time, allowing users to interpret and compare models.

3.2.6 Inference Engine

- **User-Provided Input**
Load new test data or manually enter data rows.
- **Model Output**
During inference, **all models** display their output **side-by-side on a single screen**.
This helps users compare predictions and understand differences between models.

3.2.7 Visualization System

1. Dimensionality Reduction techniques are available:
 - a. Linear methods: PCA (Principal Component Analysis), ICA (Independent Component Analysis)
 - b. Non-linear methods: UMAP, Isomap, LLE (Locally Linear Embedding), MDS (Multidimensional Scaling)

Dimensity reduction compresses features (dimensions) into 2D or 3D space. This allows a user to visually inspect the structure of the dataset, keeping the important relationships intact.

2. Unsupervised Clustering Algorithms supported:

- a. **K-Means**
- b. **DBSCAN**
- c. **BIRCH**
- d. **Agglomerative Clustering**

Without the need of specific labels, this groups data into natural clusters. This leads to the advantage of user getting informed about whether there are natural patterns of categories exist in the relevant dataset.

3. Then other important visualizations include Boxplot/Kernel density estimation for the following facts

- Individual feature Distributions
 - Since this gives a quick snapshot of each of individual variable, it gives an idea whether the dataset features need scaling, whether the data is clean or noisy.
- Feature importance chart - computed using isolation forest techniques
 - Normally Isolation forests are used for anomaly detection, but they could also be used to compute feature important scores by checking for features which contribute to most to separating points.
- Correlation heatmaps – Computed using spearman method on monotonicity
 - Spearman's method executes rank-based correlations, which are ideal for identifying monotonic relationships. This provides benefits for the user like, how features move together and further reveals redundancies in the dataset. Also helps in feature engineering as well.

3.2.8 Model Support

Each of these below models were implemented for the user to get an insight into these models

A. Regression Models

- **Linear Regression** – Models the relationship between features and a target variable using a straight line. It has the support for regularization and SGD as well. SGD

(Stochastic Gradient Descent is a fast and an iterative optimization technique to update weights whenever a dataset is large)

- **Support Vector Regression (SVR)** - This is a version of SVM (Support Vector Machine). What it does is it basically tries to fit data within a tube, oppose to a linear regression method which would minimize errors
- **Decision Tree Regressor** – Splits data into branches based on the relevant feature thresholds. At each of the leaf node, a prediction is made based on average values. Leads to reducing overfitting and thereby improves performance.
- **Random Forest Regressor** – Includes as ensemble of multiple decision tress. In each leaf node, a prediction is made based on average values. It is simple but can be prone to overfitting if not pruned.
- **KNN Regressor** – This predicts the value by averaging the target values of K closest data points. These data points are called neighbours
- **Neural Networks for Regression** – This model gives the user the ability to define a neural network specifically for regression tasks.

B. Classification Models

These include same as the above regression modes but trained differently using classification-specific loss functions. For examples,

- **Logistic regression** is a linear classification model used to compute probabilities of an outcome based on the combination of one or more independent variables, with the support for multi-class classification.
- **Support Vector Classifier** builds a hyperplane that maximizes the margin between different classes

- **Decision tree classifier** splits based on feature values, thereby creating a flowchart-type tree
- **Random Forest Classifier** uses many decision trees and takes a vote from each tree to decide the final class.
- **KNN classifier** assigns the class based on the majority class of K nearest neighbours. There is no training phase, but the predictions it makes can be slow on large datasets.
- **Neural Network Classifier** outputs probabilities for each of the class using a **softmax** function in the last layer. Ideal for multi-class classification and image, text data

Furthermore, the logit outputs of these classification models will be automatically converted to the predicted class.

3.2.9 Deployment Setup

- **Cross-Platform Packaging (Windows)**
 - Package both the frontend and the backend into a single package and allowing the frontend process to start the backend as a background process.
- **User Manual + Tutorial Slides**
 - Glossary for ML terms
 - Built-in guides for beginner users.

3.2.10 Backend Logic (Python)

In the backend of the application, we implemented the following sections and integrated them using Python as the main language.

- Preprocessing Pipelines
- Model Training APIs
- Evaluation and Metrics
- Prediction Endpoints

In the table below, these sections are further clarified as well. Each of the components is listed with their purpose and their usage in the application as well.

Components	Purpose	Usage and Benefits
1. Preprocessing Pipelines	Clean and transform raw data into a suitable format for data modelling	In the application, preprocessing pipelines handle the following key preprocessing tasks for the selected datasets - <i>Handles missing values</i> - <i>Feature Scaling</i> - <i>Outlier handling</i> - <i>Train-Test Split</i> These ensure the consistency across training/inference and automate, standardize the prep

2. Model Training APIs	Train models using relevant algorithms and parameters	The Models training APIs are implemented for the following processes, -Accept model type & configuration -Fit the relevant model to the training data -Perform cross-validation -Save trained models These processes ensure modular, reusable training and support different models through one interface
3. Evaluation & Metrics	Evaluate model performance using standard metrics for regression or classification	The evaluation and metrics component ensures below functionalities in application. - Compute metrics - Classification - Regression - Visuals This component helps a user to select the best model for a dataset Also, the component quantifies performance

4. Prediction Endpoints	Save trained models via APIs for real-time or batch inference	Prediction endpoints implement the following processes. <i>-Preprocess if needed</i> <i>-Run predictions</i> <i>-Return output as a response</i>
--------------------------------	---	---

Table 1 – Backend Components of the application

3.2.11 Synthetic Data Generation

- A section to generate synthetic data for both regression and classification
- For Regression, data can be generated with linear or non-linear interactions as well as Gaussian processes
- For the classification, data is generated to match a high-dimensional manifold, thereby capturing complex patterns in high-dimensional spaces.

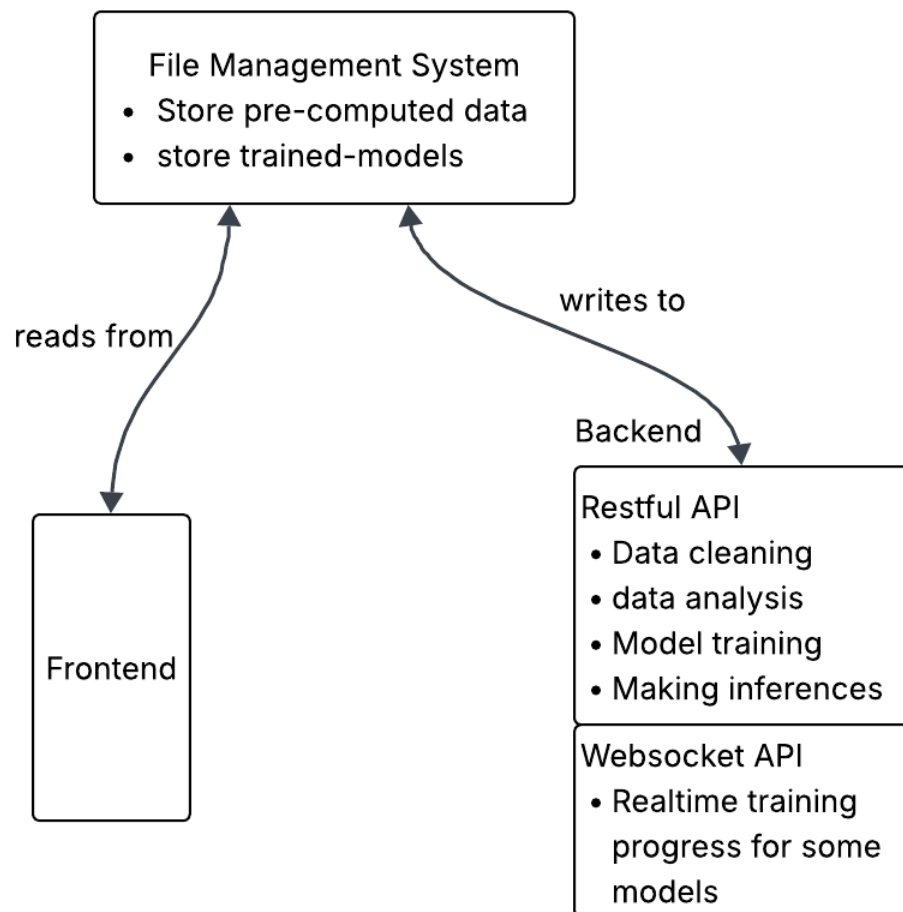


Figure 2: Basic structure of the solution

The above diagram represents the basic architecture of the application. The frontend of the system is where the user might interact and deal with machine learning models. It retrieves precomputed data and trained models from the file management system to display insights and results

The backend of the application is mainly responsible for the processing tasks, such as data cleaning, analysis of data, model training, and making predictions. A web socket API is used to provide a real-time training progress for some models as well. The backend writes all processed data and trained models to the file management system, creating a clear data flow from processing storage

4. Implementation

4.1 Technologies used

For frontend development, we chose React.js combined with Electron.js, ensuring a modern and cross-platform desktop application. However, due to the limited availability of machine learning libraries in JavaScript, we developed the backend using Python with FastAPI [8] and Uvicorn, providing a fast request-response model for communication between the frontend and backend.

In the Python backend, we implemented Auto EDA algorithms that automatically analyze datasets, apply necessary transformations, and optimize feature selection. These tasks rely on:

- Data manipulation libraries: Pandas [9], and NumPy [10]
- Statistical and machine learning libraries: Scikit-learn [11] and SciPy [12]

These algorithms operate based on user-defined configurations, though a default configuration is provided to accommodate beginners.

4.2 User Interface Design

The user interface (UI) of our machine learning GUI application introduces a beginner-friendly experience. It focuses on user-friendly navigation, real-time interaction, and educational support. this ensures that users of all technical levels can experiment with ML workflows without writing code.

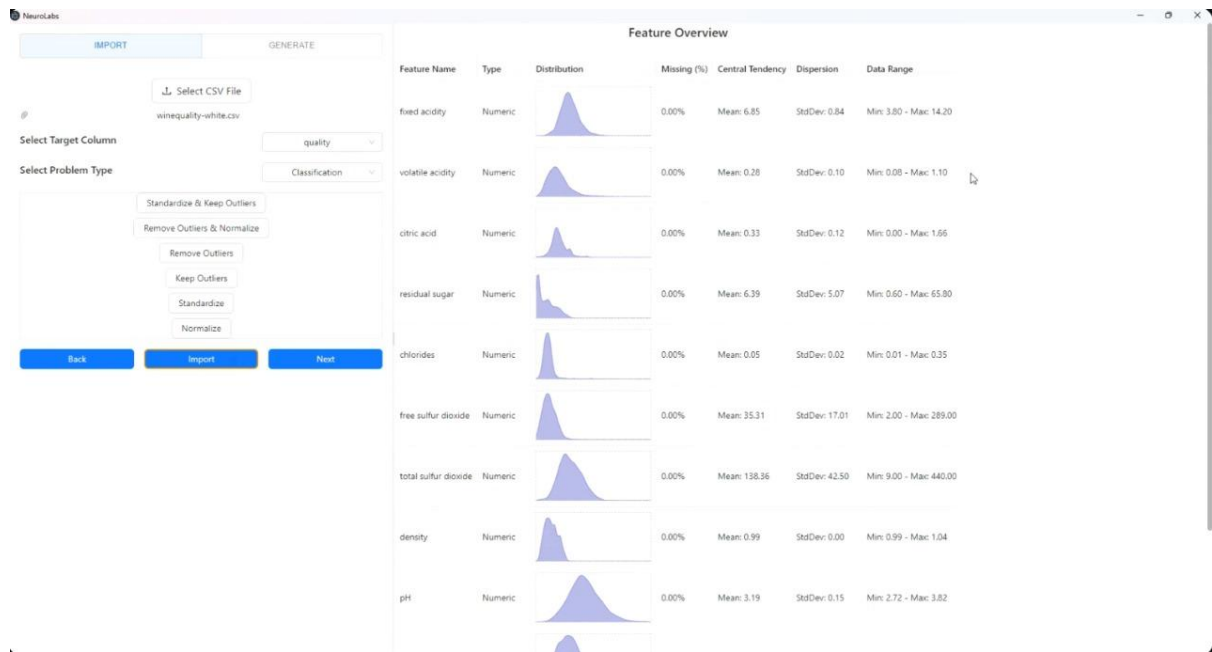


Figure 3: Data Importing & Preprocessing Module

4.3 Reports Generated

As the testing and validation phase is still ongoing, no formal reports have been generated at this stage. Once testing is complete and the system's functionality, performance, and reliability have been fully evaluated, detailed reports will be compiled. These reports will include summaries of test results, performance metrics, identified issues, and any improvements made during the validation process. Until then, report generation remains pending as part of the final evaluation workflow.

5. Discussion and Conclusion

• Discussion on the process followed during the project

Throughout the project lifecycle, we followed a modular and iterative development strategy, allowing us to build, test, and refine individual components such as the Preprocessor, Model Builder, and Inference Engine independently. This approach ensured that each section remained cohesive, testable, and educationally effective.

The decision to implement a **drag-and-drop GUI** significantly enhanced usability, allowing students to construct ML workflows visually. During testing, even users with no prior exposure to Python or machine learning concepts were able to build and evaluate basic models successfully. This confirms the effectiveness of our visual-first concept.

The integration of libraries like **scikit-learn**, **PyTorch**, and **NumPy** enabled a flexible backend capable of supporting both traditional ML and simple neural networks. Real-time feedback features such as **live metric visualizations**, confusion matrices, and ROC curves proved crucial in helping users interpret model outcomes meaningfully.

Challenges we encountered included:

- Balancing GUI simplicity with functionality.
- Ensuring the backend could respond fast enough for support real-time visualizations.

However, these challenges provided deep insight into not just ML development, but software architecture and user-experience design as well.

• Conclusions drawn

We were able to fulfil our primary objective of offering a code-free, educational platform to learn machine learning by developing this application successfully. It connected both the theory and the practice together by enabling users to:

- Explore real-world datasets
- Process data
- Train and evaluate models
- Visualize interpreted results
- Make the predictions

Critical ML principles are reinforced by this practical experience, enabling educators and students to fully interact with the data science pipeline. For non-technical users, the program can be used as a prototyping tool, a self-learning tool, or even a classroom partner.

The project also demonstrated the feasibility of using contemporary open-source stacks (**FastAPI + PyTorch + React/Electron**) to develop software that is user-friendly, scalable, and educationally valuable.

6. Future Work

- **Web-Based Application Development**

One of the primary future enhancements planned for this project is the development of a full-featured **web-based application**. This will allow users to access the system through a browser interface without needing to install any software locally.

- **Cross-Platform Compatibility**

To make the system more versatile and widely usable, another key goal is to ensure **cross-platform compatibility**. The current setup is limited to Windows, but future versions will be designed to run seamlessly across operating systems, including macOS, and Linux as well.

- **Advanced Multi-Parameter Visualizations**

Another important area of development involves the implementation of **advanced multi-parameter data visualizations**. Future versions of the system will support dynamic, interactive visualizations capable of displaying relationships between multiple features in high-dimensional datasets.

- **Automate Hypothesis Testing with Tools**

Development of an interactive tool or software module that automates:

- Choosing the correct test (z, t, paired t, etc.)
- Calculating test statistics
- Visualizing rejection regions and p-values

7. References

- [1] P. Ma et al., "MetaInsight: Automatic Discovery of Structured Knowledge for Exploratory Data Analysis," in Proceedings of the 2021 International Conference on Management of Data, 2021, pp. 241-253.
- [2] A. Kaul, S. Maheshwary and V. Pudi, "AutoLearn — Automated Feature Generation and Selection," 2017 IEEE International Conference on Data Mining (ICDM), New Orleans, LA, USA, 2017, pp. 217-226, doi: 10.1109/ICDM.2017.31.
- [3] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter, "Auto-Sklearn 2.0: Hands-free AutoML via Meta-Learning," arXiv preprint arXiv:2007.04074, 2020.
- [4] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in Advances in Neural Information Processing Systems 28 (NeurIPS), 2015, pp. 2962–2970.
- [5] E. Brochu, V. Cora, and N. de Freitas, "A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning," CoRR, vol. abs/1012.2599, 2010.
- [6] J. Demšar, T. Curk, A. Erjavec, Č. Hočevar, M. Milutinovič, M. Možina, M. Polajnar, M. Toplak, A. Starič, M. Štajdohar, L. Umek, L. Žagar, J. Žbontar, M. Žitnik, and B. Zupan, "Orange: Data Mining Toolbox in Python," Journal of Machine Learning Research, vol. 14, pp. 2349–2353, 2013.
- [7] M. R. Berthold, N. Cebon, F. Dill, T. R. Gabriel, T. Kötter, T. Meinl, P. Ohl, C. Sieb, K. Thiel, and B. Wiswedel, "KNIME: The Konstanz Information Miner," in Studies in Classification, Data Analysis, and Knowledge Organization (GfKL 2007), Springer, 2007, pp. 319–326.
- [8] Sebastián Ramírez, "FastAPI: The modern, fast (high-performance) web framework for building APIs with Python 3.7+," 2018. [Online]. Available: <https://fastapi.tiangolo.com>.

[9] Wes McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.

[10] Charles R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020.

[11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[12] Pauli Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2020.

[13] C. Foppiano, K. A. Hamilton, and M. Carugo, "Automated Model Training (AMT) GUI: An Opportunity for Integrating Machine Learning into Scientific Research," arXiv preprint arXiv:2311.13808, 2023. [Online]. Available: <https://arxiv.org/abs/2311.13808>

[14] J. J. Dudley and P. O. Kristensson, "A Review of User Interface Design for Interactive Machine Learning," *ACM Transactions on Interactive Intelligent Systems*, vol. 8, no. 2, pp. 1-37, June 2018. [Online]. Available: A Review of User Interface Design for Interactive Machine Learning | ACM Transactions on Interactive Intelligent Systems

[15] F. J. Bernardo, M. Zbyszynski, R. Fiebrink, and M. Grierson, "Interactive Machine Learning for End-User Innovation," *Proceedings of the 2017 ACM Conference on Creativity and Cognition*, pp. 1-12, June 2017. [Online]. Available: Interactive Machine Learning for End-user Innovation

[16] O. Abdel-Hamid, A.-r. Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional Neural Networks for Speech Recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 10, pp. 1533–1545, Oct. 2014, doi: 10.1109/TASLP.2014.2339736.

[17] X. Zhang, J. Zhao, and Y. LeCun, "Character-level Convolutional Networks for Text Classification," in *Advances in Neural Information Processing Systems (NIPS)*, Montreal,

Project Report - Group 10

Canada, Dec. 2015. [Online]. Available: Character-level Convolutional Networks for Text Classification

Project Supervisor :

.....

(Signature)

03/26/2024 Date